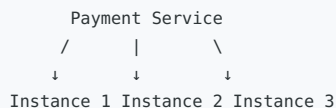


Load Balancing Strategies

1. What is Load Balancing?

Suppose you have one service with multiple instances:



Thousands of requests arrive:



The load balancer distributes requests among available instances.

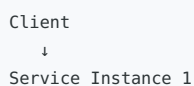
Why?

Main goals:

- distribute traffic
- improve availability
- use multiple instances efficiently
- avoid overloading one instance
- support horizontal scaling
- route away from unhealthy instances

2. Basic Architecture

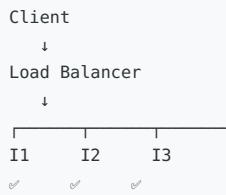
Without load balancing:



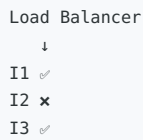
If Instance 1 goes down:



With load balancing:



If I2 fails:

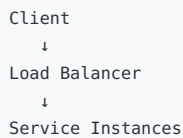


Traffic can be sent to healthy instances.

3. Two Major Types

Server-Side Load Balancing

The client talks to a load balancer.



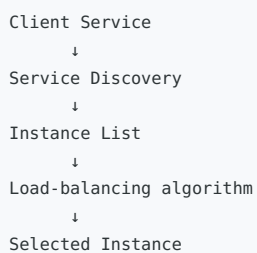
Examples:

- AWS Application Load Balancer
- NGINX
- HAProxy
- Kubernetes Service/load-balancing mechanisms

The client doesn't choose the backend instance.

Client-Side Load Balancing

The client knows or discovers the available instances and selects one.



Spring Cloud LoadBalancer is an example of a client-side load-balancing component.

4. Common Load-Balancing Strategies

These are the main ones you should know.

4.1 Round Robin

Requests are distributed sequentially.

```
Request 1 → Instance 1  
Request 2 → Instance 2  
Request 3 → Instance 3  
Request 4 → Instance 1  
Request 5 → Instance 2
```

Pattern:

```
I1 → I2 → I3 → I1 → I2 → I3
```

Advantages

Simple and predictable.

Disadvantages

Assumes instances have roughly similar capacity and requests have roughly similar cost.

If one request takes 30 seconds and another takes 5 ms, simple round robin doesn't account for that.

5. Weighted Round Robin

Give instances different weights.

Suppose:

```
Instance 1 → weight 5  
Instance 2 → weight 3  
Instance 3 → weight 2
```

Instance 1 gets roughly more requests.

Useful when:

```
I1 = 8 CPU  
I2 = 4 CPU  
I3 = 2 CPU
```

The stronger machine can handle more traffic.

6. Random

Select an instance randomly.

```
Request
↓
Random selection
↓
I1 / I2 / I3
```

Very simple, but doesn't explicitly account for current load.

7. Least Connections

Send the next request to the instance with the fewest active connections.

Example:

```
I1 → 10 connections
I2 → 3 connections
I3 → 7 connections
```

Next request:

```
→ I2
```

Useful when requests have varying durations.

8. Weighted Least Connections

Combination of:

```
current connections
+
instance capacity/weight
```

Useful when instances have different capabilities.

9. IP Hash / Consistent Hashing

The client's IP or another key is hashed to determine the destination.

```
Client A → Instance 2
Client B → Instance 1
Client C → Instance 3
```

The same client/key tends to reach the same instance.

This can be useful for **session affinity/stickiness**.

However, modern stateless applications often try to avoid depending on sticky sessions.

10. Consistent Hashing

A more specialized hashing strategy that minimizes remapping when nodes are added or removed.

Useful for systems such as:

- distributed caches
- sharded systems
- partitioned workloads

Example:

Hash Ring

I1

I3

I2

A key maps to a location on the ring and is assigned to the appropriate node.

You should know the concept, but it is less important than Round Robin and Least Connections for a typical 3-year Spring Boot interview.

11. Least Response Time

Select an instance based partly on response time.

Conceptually:

I1 → 100 ms

I2 → 500 ms

I3 → 150 ms

Next request → I1

This can adapt to current performance, but requires measuring latency and maintaining relevant state.

12. Health Checks

Load balancing isn't just about selecting an instance.

It should also determine:

Is the instance healthy enough to receive traffic?

Example:

```
I1 → HEALTHY
I2 → UNHEALTHY
I3 → HEALTHY
```

Traffic:

```
      Load Balancer
     /      \
    ↓        ↓
   I1       I3
```

I2 is temporarily removed from the active pool.

13. Active vs Passive Health Checking

Active

Load balancer periodically calls a health endpoint:

```
GET /health
```

Passive

The system observes actual request failures/timeouts and adjusts routing accordingly.

Many real systems use combinations of health signals.

14. Load Balancer Does NOT Guarantee Fault Tolerance

Suppose all instances are unhealthy:

```
I1 x
I2 x
I3 x
```

The load balancer can't magically fix the service.

You still need:

- redundancy
 - autoscaling
 - timeouts
 - retries
 - circuit breakers
 - proper capacity planning
 - monitoring
-

15. Spring Cloud LoadBalancer

Spring Cloud provides **Spring Cloud LoadBalancer** for client-side load balancing.

Suppose service discovery gives:

```
PAYMENT-SERVICE
├─ 10.0.0.1:8080
├─ 10.0.0.2:8080
└─ 10.0.0.3:8080
```

A client can use the logical service name:

```
payment-service
```

and the load-balancing component selects an instance.

Conceptually:

```
Order Service
  ↓
payment-service
  ↓
Spring Cloud LoadBalancer
  ↓
Instance selection
  ↓
Payment Service
```

16. Eureka + Load Balancer

These are often confused.

Eureka:

Where are the service instances?

Load Balancer:

Which instance should I call?

Example:

```
      Eureka
      ↓
Available instances
  /      |      \
 I1      I2      I3
      ↓
    Load Balancer
      ↓
    Selected I2
```

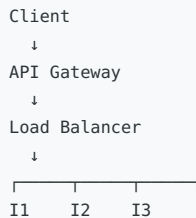
So:

```
Service Discovery
→ instance discovery
```

Load Balancing
→ instance selection

17. API Gateway + Load Balancing

These can also work together.



Gateway handles things like:

- routing
- authentication policies
- rate limiting
- filtering

Load balancer distributes requests among service instances.

18. Example Using Feign

Suppose:

```
@FeignClient(name = "payment-service")
public interface PaymentClient {

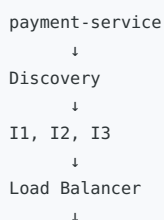
    @GetMapping("/payments/{id}")
    PaymentResponse getPayment(
        @PathVariable String id);
}
```

The logical name is:

payment-service

In a service-discovery/load-balancing setup, Spring Cloud infrastructure can resolve that logical name to an available instance.

Conceptually:



19. Why Load Balancing Matters for Microservices

Suppose Payment Service gets high traffic.

You scale:

```
1 instance
  ↓
3 instances
  ↓
10 instances
```

The client shouldn't need to know each IP manually.

Instead:

```
payment-service
  ↓
load balancing
  ↓
available instances
```

This supports **horizontal scaling**.

20. Scaling Example

Suppose each instance handles 100 requests/sec.

One instance:

```
100 req/sec
```

Three instances:

```
I1 → 100
I2 → 100
I3 → 100
  ↓
~300 req/sec capacity
```

This is simplified; real throughput depends on workload and bottlenecks.

21. What If Instances Have Different Capacities?

Suppose:

```
I1 → 8 CPU
I2 → 4 CPU
I3 → 2 CPU
```

Simple round robin:

```
I1  
I2  
I3  
I1  
I2  
I3
```

may not be ideal.

A weighted strategy could allocate approximately:

```
I1 → 5 requests  
I2 → 3 requests  
I3 → 2 requests
```

This is why the choice of algorithm depends on the environment.

22. Sticky Sessions

Suppose a user's session exists only in Instance 1:

```
User A → I1
```

If the next request goes to I2:

```
User A → I2
```

the session isn't available there.

Sticky sessions can ensure:

```
User A → I1  
User A → I1  
User A → I1
```

But in modern microservices, it's generally preferable to make services **stateless** and store session/state externally when needed.

That makes scaling and failover easier.

23. Load Balancing + Retry

Be careful here.

Suppose:

```
Request → Instance 1  
      ↓  
    timeout
```

The load balancer/retry mechanism might send it to Instance 2.

That can be useful for idempotent operations:

GET

But for state-changing operations:

POST payment

retrying against another instance can potentially duplicate the business operation unless idempotency is enforced.

Again:

Load balancing doesn't make retries automatically safe.

24. Load Balancing + Circuit Breaker

These complement one another.

```
graph TD
  Caller --> CB[Circuit Breaker]
  CB --> LB[Load Balancer]
  LB --> SI[Service instances]
```

Possible flow:

```
graph TD
  A[Service instance repeatedly failing] --> B[Load balancer stops routing to unhealthy instance]
```

and/or:

```
graph TD
  A[Dependency/service repeatedly failing] --> B[Circuit breaker OPEN]
```

Exact behavior depends on the implementation.

25. Common Interview Questions

Basic

1. What is load balancing?
2. Why is load balancing needed?
3. What is horizontal scaling?
4. Round Robin?
5. Weighted Round Robin?

6. Least Connections?
7. Random?
8. Sticky sessions?

Microservices

9. Client-side vs server-side load balancing?
10. Eureka vs Load Balancer?
11. API Gateway vs Load Balancer?
12. How does Feign use service discovery?
13. How do you route traffic away from unhealthy instances?
14. What happens when one service instance crashes?

3-year level

15. What load-balancing strategy would you choose and why?
16. Round Robin vs Least Connections?
17. How would you handle instances with different capacity?
18. How does load balancing work with autoscaling?
19. How does retry interact with load balancing?
20. How does circuit breaker interact with load balancing?
21. How would you design load balancing for a high-traffic payment service?
22. What happens when all instances become unhealthy?

26. Interview Scenario

Interviewer:

“You have three instances of Payment Service. How will requests be distributed?”

You could answer:

“A load balancer maintains or obtains the available service instances and selects one according to its configured strategy, such as Round Robin or Least Connections. It should also consider health status so unavailable instances don’t receive traffic. In a Spring Cloud environment, service discovery such as Eureka can provide the instance information while Spring Cloud LoadBalancer can handle client-side instance selection. The exact architecture could instead use infrastructure-side load balancing such as an AWS load balancer or Kubernetes.”

27. Interview Scenario — Which Strategy?

“Would you always use Round Robin?”

No.

A strong answer:

"Round Robin is simple and works well when instances have similar capacity and requests have reasonably similar processing cost. If request durations vary significantly, Least Connections or a latency/load-aware strategy may be more appropriate. If instances have different capacities, weighted strategies can be useful."

28. Interview Scenario — Service Discovery

"What is the difference between Eureka and load balancing?"

Strong answer:

"Eureka is a service registry that helps discover service instances. Load balancing is the process of selecting which available instance should receive a request. They are complementary: discovery provides the instance list, while the load-balancing component chooses an instance."

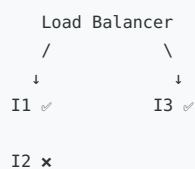
29. Interview Scenario — Payment System

Suppose:

```
Payment Service
├─ I1
├─ I2
└─ I3
```

and I2 is failing.

A good architecture:



But there is an important distinction:

*If the request already reached I2 and timed out during payment processing, simply sending the same payment request to I1 is **not automatically safe**.*

You first need to consider:

Was payment actually processed?

↓
idempotency
↓
transaction state
↓
reconciliation

That is where load balancing meets real backend/payment design.

30. BPS Questions — Investigate Your Codebase

Now inspect your BPS architecture:

1. Are there multiple BPS instances running in an environment?
 2. What performs load balancing for BPS?
 3. Is it an infrastructure load balancer, API gateway, Kubernetes, or Spring Cloud LoadBalancer?
 4. Does BPS use Eureka/Consul/service discovery?
 5. How are Hydra/Chase endpoints selected?
 6. Does BPS itself load-balance calls to any downstream service?
 7. What happens if one BPS instance becomes unhealthy?
 8. How are health checks performed?
 9. Is BPS stateless?
 10. What happens if the same payment request reaches two BPS instances concurrently?
-

Interview-ready summary

“Load balancing distributes traffic across multiple service instances to improve availability, utilization and scalability. Common strategies include Round Robin, Weighted Round Robin, Least Connections, Random and hashing-based approaches. Load balancing can be server-side, where a dedicated infrastructure component selects the backend instance, or client-side, where the calling service selects an instance from a discovered list. In Spring Cloud, service discovery can provide the available instances and Spring Cloud LoadBalancer can select an instance. Eureka and load balancing have different responsibilities: Eureka answers where service instances are, while load balancing decides which instance should receive a request. The strategy should depend on instance capacity, request characteristics and workload, rather than assuming Round Robin is always best.”

Memorize this

Service Discovery

→ "What instances are available?"

Load Balancer

→ "Which instance should receive this request?"

API Gateway

→ "Where does the external request enter and how is it routed?"

Health Check

→ "Is this instance safe to send traffic to?"

The highest-value interview topics here are **Round Robin vs Least Connections, client-side vs server-side load balancing, Eureka + Spring Cloud LoadBalancer, health checks, and how retries can become dangerous for state-changing/payment operations.**